Hướng dẫn chỉnh sửa Olympia Controller

Đổi cơ chế vận hành, thêm animation và thêm tính năng cho bộ điều khiển đồ hoạ "Đường lên đỉnh Olympia" (HTML5)



Tài liệu này dành cho người muốn đổi cách controller vận hành, thêm hiệu ứng hoặc thêm tính năng. Bạn cần biết JavaScript ở mức đọc được hàm, `if/else` và biến. Nếu đã từng sửa bộ WWTBAM HTML5 Controller (Olga), cách tổ chức ở đây gần như giống hệt: hàm toàn cục, mọi trạng thái nằm trong `window.GameVariables`, mọi phím nằm trong `KeyboardController.js`. Chương 11 đối chiếu chi tiết hai dự án.

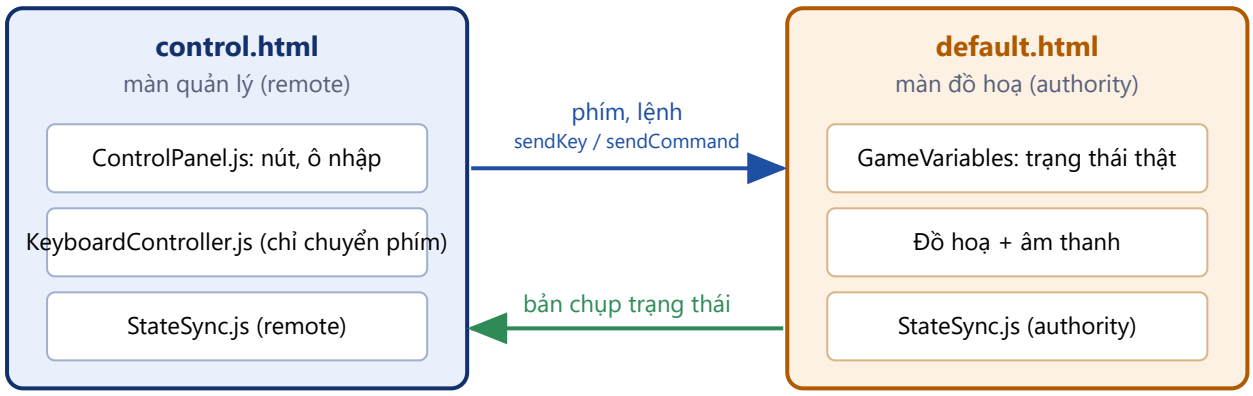
Mục lục

1. Chạy và thử nghiệm
2. Bản đồ các file
3. Ba điều cần nắm trước khi sửa
4. Sửa cấu hình (không đụng logic)
5. Sửa cơ chế của một vòng thi
6. Thêm animation
7. Thêm tính năng mới
8. Quy tắc để không làm hỏng thứ khác
9. Kiểm tra sau khi sửa
10. Lỗi thường gặp
11. Đối chiếu với WWTBAM HTML5 Controller gốc

1. Chạy và thử nghiệm

Controller gồm hai trang, phục vụ qua IIS:

Trang	Vai trò
`default.html`	Màn đồ hoạ, đưa lên máy chiếu / phát sóng. Là trang duy nhất phát âm thanh và là nơi giữ trạng thái thật của buổi thi.
`control.html`	Màn quản lý của người điều khiển. Chỉ hiển thị trạng thái và gửi phím / lệnh sang `default.html`.



Hình 1. Hai trang và chiều dữ liệu giữa chúng

Hai trang nói chuyện qua BroadcastChannel và localStorage, nên phải mở cùng một trình duyệt trên cùng một máy. Chỉ mở một tab `default.html`; mở hai tab thì âm thanh bị lặp (trang sẽ hiện cảnh báo đỏ).

Khi thử nghiệm, mở `default.html`, bấm F12 để mở Console. Ba lệnh gõ thẳng vào Console:

```
pressKey('ArrowLeft')                    // giống hệt bấm phím ← trên trang
window.GameVariables                     // xem toàn bộ trạng thái hiện tại
window.GameVariables.VcnvSequenceCounter  // vòng VCNV đang ở bước số mấy
```

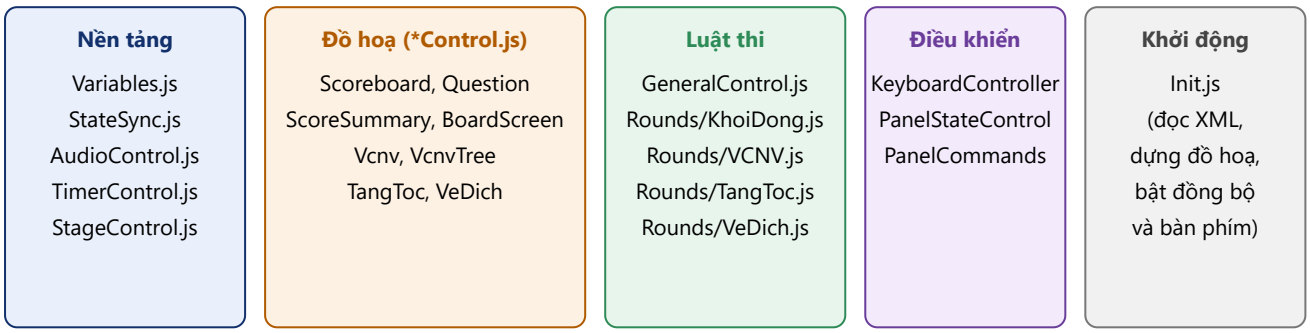
Sau khi sửa file, tải lại trang bằng Ctrl+F5 để trình duyệt không dùng bản cũ trong bộ nhớ đệm.

2. Bản đồ các file

Thứ tự dưới đây cũng là thứ tự nạp trong `default.html`.

File	Chứa gì	Khi nào cần mở
`Scripts/Variables.js`	Cấu hình (điểm, thời gian, âm thanh, file dữ liệu) và toàn bộ biến trạng thái	Đổi điểm / thời gian / âm thanh; thêm biến mới
`Scripts/StateSync.js`	Đồng bộ hai trang	Gần như không bao giờ
`Scripts/AudioControl.js`	7 kênh âm thanh, nhạc nền có vòng lặp	Thêm cách phát âm thanh mới
`Scripts/TimerControl.js`	Đồng hồ chung (`startClock` , `stopClock`)	Hiếm khi
`Scripts/StageControl.js`	Nền, thẻ tên vòng, hàm dựng phần tử (`newElement` , `newImage` , `newGlare`)	Thêm thẻ tên vòng
`Scripts/ScoreboardControl.js`	4 thanh tên thí sinh, cộng / đặt điểm	Đổi cách hiện thanh tên, điểm
`Scripts/QuestionControl.js`	Thanh câu hỏi Khởi động, đồng hồ 3 giây	Đổi thanh câu hỏi KĐ
`Scripts/ScoreSummaryControl.js`	Bảng tổng kết điểm (phím M)	
`Scripts/ScoreBoardScreenControl.js`	Bảng kết quả hiện tại (F6)	
`Scripts/VcnvControl.js`	Đồ hoạ VCNV: ô chữ, ảnh, thanh câu hỏi	
`Scripts/VcnvTreeControl.js`	Cây đáp án (dùng chung cho VCNV và Tăng tốc)	
`Scripts/TangTocControl.js`	Khung câu hỏi Tăng tốc	
`Scripts/VeDichControl.js`	Màn chọn gói, thanh câu hỏi, ngôi sao Về đích	
`Scripts/GeneralControl.js`	`switchRound()` : dọn vòng cũ, vào vòng mới	Thêm vòng mới
`Scripts/Rounds/KhoiDong.js` , `VCNV.js` , `TangToc.js` , `VeDich.js`	Các hàm bước của từng vòng (hiện câu hỏi, chấm, cộng điểm...)	Đổi luật, đổi trình tự
`Scripts/KeyboardController.js`	Mọi phím: phím nào, ở bước nào, gọi hàm bước nào	Đổi phím, thêm bước, thêm phím
`Scripts/PanelStateControl.js`	Chuyển trạng thái sang dạng màn quản lý đọc	Muốn màn quản lý thấy thêm một thông tin
`Scripts/PanelCommands.js`	Các lệnh nút bấm trên màn quản lý gửi sang	Thêm nút mới trên màn quản lý
`Scripts/Init.js`	Khởi động trang, đọc các file XML	Thêm file câu hỏi mới
`Scripts/ControlPanel.js`	Toàn bộ màn quản lý	Thêm nút / ô hiển thị trên màn quản lý
`Scripts/QuestionLoader.js`	Để trống (nội dung đã chuyển vào `Init.js`)	Không

`Styles/olympia.css`	Vị trí, kích thước, hiệu ứng của mọi đồ hoạ	Mọi thay đổi về hình ảnh và animation
`Styles/control.css`	Giao diện màn quản lý	
`Questions/*.xml`	Thí sinh và câu hỏi (mỗi file có chú thích cách điền ở đầu)	Soạn đề
`Sounds/` , `024/`	Âm thanh, hình ảnh	



Hình 2. Các nhóm file theo thứ tự nạp

3. Ba điều cần nắm trước khi sửa

3.1. Mọi trạng thái là một biến phẳng trong `window.GameVariables`

Không có biến ẩn trong hàm, không có object lồng nhau cho từng vòng. Muốn biết "VCNV đang mở hàng nào" thì đọc `window.GameVariables.VcnvCurrentRow`; muốn biết "Về đích ai đang đứng trên sân khấu" thì đọc `window.GameVariables.VeDichSeat`. Tất cả được khai báo trong `Variables.js`, chia khối như Olga: *Boolean Variables* (tên bắt đầu bằng `Is`), *Number Variables*, *String Variables*, *Array / Object Variables*, *Audio Variables*, *Interval / Timeout Variables*.

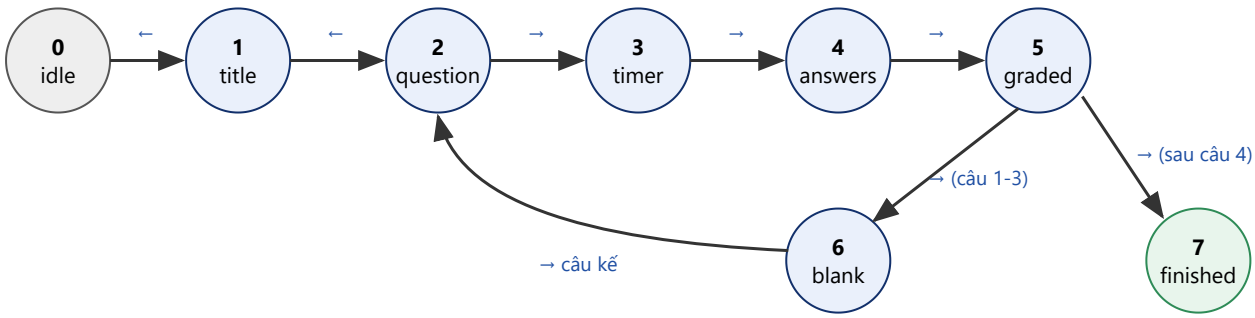
Tên biến bắt đầu bằng tên vòng: `KhoiDong...`, `Vcnv...`, `TangToc...`, `VeDich...`.

3.2. Mỗi vòng có một bộ đếm bước

Giống `QuestionSequenceCounter` của Olga. Bộ đếm là một con số; mỗi số là một bước của vòng. Bảng ý nghĩa nằm ngay cạnh bộ đếm trong `Variables.js` và được chép lại ở đầu `KeyboardController.js`:

```
KĐ    0 idle 1 title 2 ready 3 set 4 question 5 done 6 finished
VCNV  0 idle 1 title 2 board 3 rowSelected 4 rowQuestion 5 rowTimer 6 rowAnswers 7 rowGraded
      8 rowBack 9 guessing 10 finalWindow 11 finished 12 done
TT    0 idle 1 title 2 question 3 timer 4 answers 5 graded 6 blank 7 finished 8 done
VĐ    0 idle 1 title 2 stageCall 3 choose 4 ready 5 question 6 timer 7 steal 8 stealAnswer
      9 answered 10 blank 11 finished 12 done
```

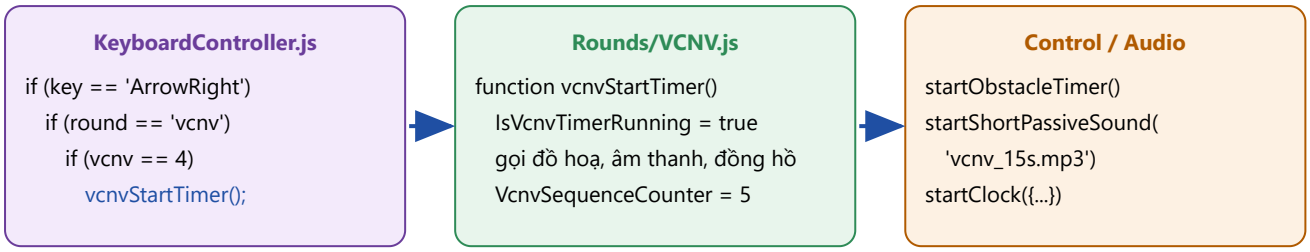
Các mảng `KhoiDongPhaseNames`, `VcnvPhaseNames`... trong `Variables.js` chứa đúng các tên trên theo thứ tự số. Màn quản lý in trạng thái dựa vào các tên này. Hình 3 vẽ bộ đếm của vòng Tăng tốc (vòng có ít bước nhất) cùng phím chuyển giữa các bước.



Phím 1-4 đánh dấu đúng ở bước 2, 3, 4. Phím B xem lại khung câu hỏi ở bước 4, 5. ← ở bước 7 → 8 (done) → sang Về đích.

Hình 3. Bộ đếm `TangTocSequenceCounter` và các phím chuyển bước

3.3. Một phím đi qua ba tầng



phím → ở bước nào? → hàm bước (đổi biến, đặt bộ đếm) → hàm đồ hoạ / âm thanh → changed() cập nhật màn quản lý

Hình 4. Bấm → khi VCNV đang ở bước 4 (câu hỏi hàng đang hiện)

1. `KeyboardController.js` có đoạn `else if (vcnv == 4) { vcnvStartTimer(); }` nằm trong khối `if (key == 'ArrowRight')`, nhánh `round == 'vcnv'`.
2. `vcnvStartTimer()` trong `Rounds/VCNV.js` bật cờ `IsVcnvTimerRunning`, gọi `startObstacleTimer()` (đồ hoạ), `startShortPassiveSound('vcnv_15s.mp3')` (âm thanh), `startClock(...)` (đồng hồ), rồi đặt `VcnvSequenceCounter = 5`.
3. Sau mỗi phím, `KeyboardController.js` gọi `window.GameVariables.changed()` để màn quản lý cập nhật.

Muốn tìm "phím X ở vòng Y làm gì", mở `KeyboardController.js`, tìm `key == 'X'` rồi tìm `round == 'y'` bên trong.

Nút trên màn quản lý đi đúng đường này: nút gửi lệnh (ví dụ `vcnvKey` với phím `ArrowRight`), `PanelCommands.js` nhận và gọi `Keyboard.dispatchTo('vcnv', 'ArrowRight')`, tức là chạy đúng nhánh như khi bấm phím thật.

4. Sửa cấu hình (không đụng logic)

Mọi thứ trong chương này nằm ở khối *Initialization Variables* đầu `Variables.js`. Mỗi dòng có chú thích.

Điểm số: `gameVariables.Points`. Ví dụ muốn Khởi động lượt chung trả lời sai bị trừ 10 thay vì 5, đổi `KD_WRONG_COMMON: -5` thành `-10`.

Thời gian: `gameVariables.Timings`. `VCNV_DECIDE: 15` là 15 giây cho mỗi hàng VCNV; `VE_DICH_ANSWER: { 20: 15, 30: 20 }` là thời gian trả lời theo giá trị câu Về đích. Thời gian từng câu Tăng tốc có thể ghi đè bằng thuộc tính `seconds` trong `tangtoc.xml`.

> Nhạc đồng hồ là file có độ dài cố định. Đổi `VCNV_DECIDE` thành 20 thì thanh thời gian chạy 20 giây nhưng `vcnv_15s.mp3` vẫn chỉ dài 15 giây. Muốn khớp thì thay file nhạc.

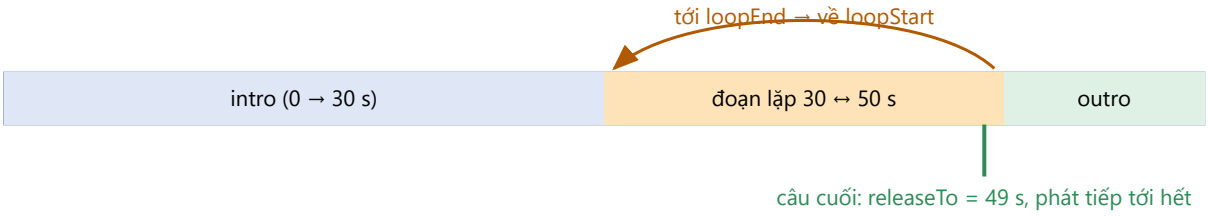
Đổi một file âm thanh: chép file mới vào `Sounds/` với đúng tên cũ (ví dụ `kd_correct.mp3`). Không phải sửa mã.

Thêm một file âm thanh mới: chép vào `Sounds/`, thêm một dòng vào `gameVariables.SoundMap`:

```
'tb_start.mp3': true,                    /* Câu hỏi phụ: bắt đầu */
```

File sẽ tự xuất hiện trên bảng âm thanh của màn quản lý. Muốn nút có tên tiếng Việt, thêm vào `soundLabels` trong `ControlPanel.js`. Muốn phát nó ở một bước nào đó, xem mục 7.4.

Nhạc nền có đoạn lặp (hiện chỉ có `kd_60s.mp3`): `gameVariables.BedLoops`. `loopStart` / `loopEnd` là giây trong file; khi phát tới `loopEnd` nhạc nhảy về `loopStart`. Khi hiện câu hỏi cuối của lượt, `releaseLongPassiveSound()` nhảy tới `releaseTo` để phần kết của bản nhạc được phát.



Hình 5. Nhạc nền `kd_60s.mp3` theo `BedLoops`

Đề thi và tên thí sinh: sửa `Questions/*.xml` (không phải mã). Tên gõ trên màn quản lý được nhớ trên máy và ưu tiên hơn `players.xml`; nút "đặt lại tên" trên màn quản lý quay về tên trong XML.

5. Sửa cơ chế của một vòng thi

5.1. Đổi âm thanh hoặc hình ảnh ở một bước

Tìm hàm bước tương ứng trong `Scripts/Rounds/`. Ví dụ muốn Tăng tốc dùng âm thanh chấm điểm riêng thay vì mượn của VCNV: mở `Rounds/TangToc.js`, tìm `tangTocGradeQuestion`, đổi `'vcnv_row_correct.mp3'` / `'vcnv_row_wrong.mp3'` thành tên file mới (nhớ thêm file vào `SoundMap`).

Chọn đúng hàm phát âm thanh theo kênh (bảng ở đầu `AudioControl.js`):

Hàm	Kênh	Dùng cho
`startGeneralSound(file)`	StingerSound	Hiệu ứng ngắn: bắt đầu vòng, mở câu hỏi. Nhiều tiếng có thể chồng lên nhau
`startLongActiveSound(file)`	ResultSound	Đúng / sai. Tiếng mới cắt tiếng cũ trên kênh này
`startShortPassiveSound(file)` / `stopShortPassiveSound()`	TimerSound	Nhạc đồng hồ của một câu
`startLongPassiveSound(file)` / `stopLongPassiveSound()`	RoundBedSound	Nhạc nền cả lượt, theo `BedLoops`
`playBuzzerSound()`	BuzzerSound	Chuông `kd_buzz.mp3`
`startLifelineActiveSound(file)`	StarSound	Ngôi sao hy vọng
`startLifelinePassiveSound(file)`	AmbientSound	Nhạc tổng kết điểm
`stopAllSounds()`	tất cả	

5.2. Đổi phím của một thao tác

Mở `KeyboardController.js`, tìm khối của phím cũ, chuyển nhánh sang khối của phím mới. Ví dụ muốn đặt Ngôi sao hy vọng bằng phím N thay vì S: xoá nhánh trong `else if (lower == 's')`, thêm khối mới:

```
else if (lower == 'n') // key 'n' for the Ngôi sao hy vọng (VĐ)
{
        if (round == 'vd') { veDichPlaceStar(); return true; }
}
```

Trước khi chọn phím, kiểm tra bảng phím ở đầu file để không trùng phím chung (M, Z, X, 0, dấu chấm, F1 đến F6). Nhớ sửa luôn bảng chú thích ở đầu file. Nút trên màn quản lý gửi phím qua thuộc tính `data-key` trong `control.html`; tìm `data-key="s"` và đổi theo.

5.3. Đổi điều kiện của một bước

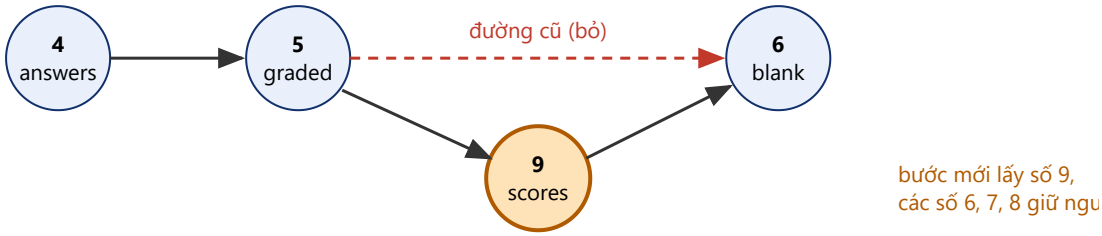
Mỗi nhánh trong `KeyboardController.js` là một câu `if` bằng số bước. Ví dụ hiện tại ở Về đích, sau khi thí sinh trả lời sai và 5 giây giành quyền đang chạy, → chỉ đóng câu khi chưa ai bấm chuông:

```
else if (vd == 7) { if (window.GameVariables.VeDichStealSeat === null) { veDichCloseQuestion(); }
}
```

Muốn cho phép → đóng câu cả khi đã có người bấm, bỏ điều kiện bên trong. Mọi điều kiện kiểu này nằm ở `KeyboardController.js` hoặc ở dòng đầu của hàm bước.

5.4. Thêm một bước mới vào giữa trình tự

Đây là thay đổi dễ gây lỗi nhất, vì các con số bước xuất hiện ở nhiều chỗ. Cách an toàn: không đánh số lại các bước cũ. Con số chỉ là tên gọi, không bắt buộc theo thứ tự thời gian, nên bước mới lấy số trống tiếp theo (Hình 6).



Hình 6. Chèn bước "scores" giữa bước 5 và 6 của Tăng tốc

Ví dụ: ở Tăng tốc, muốn sau khi chấm điểm (bước 5) có thêm một bước hiện bảng điểm trước khi cất cây đáp án.

1. `Variables.js`: thêm tên vào cuối `TangTocPhaseNames` (vị trí số 9) và ghi vào bảng chú thích:

```
gameVariables.TangTocPhaseNames = ['idle', 'title', 'question', 'timer', 'answers', 'graded',
'blank', 'finished', 'done', 'scores'];
```

2. `Rounds/TangToc.js`: viết hàm bước, cuối hàm đặt bộ đếm:

```
function tangTocShowScores() {
        hideAnswerTree();
        showBoardScreen();
        window.GameVariables.TangTocSequenceCounter = 9;
}
```

3. `KeyboardController.js`, khối `ArrowRight`, nhánh `round == 'tt'`: bước 5 giờ đi tới bước 9, bước 9 mới cất cây:

```
else if (tt == 5) { tangTocShowScores(); }
else if (tt == 9) { hideBoardScreen(); tangTocClearTree(); }
```

4. Tìm mọi chỗ khác dùng số bước của vòng này để chắc chắn bước mới không bị bỏ sót. Với Tăng tốc là `tangTocCanMark()` (bước 2 đến 4), `tangTocToggleReview` và khối phím B (bước 4, 5), `areRoundGraphicsOnScreen()` trong `ScoreSummaryControl.js` (từ bước 7 trở lên coi như đã xong vòng). Cách tìm nhanh nhất: tìm `TangTocSequenceCounter` và `tt ==` trong thư mục `Scripts`.
5. (Không bắt buộc) Màn quản lý tự in tên `scores`. Muốn in tiếng Việt, thêm `scores: '...'` vào `ttPhaseText` trong `ControlPanel.js`. Các nút nhấp nháy "phím nên bấm tiếp" trên màn quản lý tính trong hàm `ttKeys()` cũng của file này.

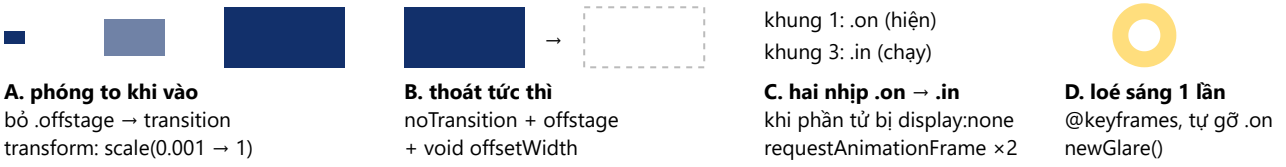
> Ví dụ trên có một lỗi cố ý để minh hoạ: F6 đang mở thì mọi phím khác bị khoá (xem đầu `Keyboard.dispatch`), nên → ở bước 9 sẽ không chạy. Muốn dùng bảng điểm ở giữa vòng, phải làm bảng riêng hoặc bỏ khoá đó. Đọc kỹ các điều kiện chặn phím trước khi dùng lại một đồ hoạ có sẵn.

6. Thêm animation

6.1. Nguyên tắc của dự án

- Mọi hiệu ứng viết bằng CSS (`transition` hoặc `@keyframes`) trong `Styles/olympia.css`. JavaScript chỉ bật / tắt class.
- Không thêm hiệu ứng mờ dần hay trượt ngoài những cái đã có, trừ khi người phụ trách đồ hoạ yêu cầu. Phần lớn đồ hoạ vào / ra tức thì, giống bản phát sóng gốc.
- Thoát luôn tức thì, kể cả với đồ hoạ vào có hiệu ứng.
- Sân khấu cố định 1920×1080, mọi phần tử định vị tuyệt đối (`position: absolute`, `left`, `top` tính bằng px).

6.2. Bốn mẫu có sẵn



Hình 7. Bốn mẫu animation có sẵn trong dự án

Mẫu A: phóng to khi vào bằng class `offstage` (thanh câu hỏi Khởi động, bảng tổng kết). CSS đặt trạng thái hiện là mặc định và trạng thái ẩn trong `.offstage`; bỏ class thì `transition` chạy:

```
.questionBarDiv {
        transform-origin: 0 0;                    /* phóng từ góc trên trái */
        transform: scale(1);
        transition: transform 900ms cubic-bezier(0.16, 0.84, 0.44, 1);
}
.questionBarDiv.offstage { transform: scale(0.001); opacity: 0; }
.questionBarDiv.noTransition { transition: none; }
```



Hình 8. Thanh câu hỏi Khởi động (`O24/Warm-up/question_bar.png`) dùng mẫu A, phóng từ góc trên trái

Mẫu B: thoát tức thì (đi kèm mẫu A). Tắt transition, ẩn, ép trình duyệt vẽ lại, rồi bật lại transition cho lần hiện sau:

```
function hideQuestionBar() {
        var root = document.querySelector('.questionBarDiv');
        if (!root) { return; }
        root.classList.add('noTransition');
        root.classList.add('offstage');
        void root.offsetWidth;                    /* dòng này bắt buộc, thiếu thì hiệu ứng vẫn
chạy khi ẩn */
        root.classList.remove('noTransition');
}
```

Mẫu C: hai nhịp `.on` rồi `.in` (khung Tăng tốc, nhóm câu hỏi Về đích, cây đáp án). Dùng khi phần tử bị `display: none` lúc ẩn: `.on` cho hiện ra, khung hình sau mới thêm `.in` để transition có điểm bắt đầu. Nếu thêm cả hai cùng lúc, trình duyệt bỏ qua hiệu ứng.

```
part.classList.add('on');
requestAnimationFrame(function () { requestAnimationFrame(function () { part.classList.add('in');
}); });
```

Mẫu D: loé sáng chạy một lần bằng `@keyframes` (ánh loé `glare.png`). Tạo bằng `newGlare(className, parent)` trong `StageControl.js`; hàm này tự gỡ class `.on` khi hiệu ứng kết thúc để lần sau thêm `.on` là chạy lại được. Muốn loé: xoá `.on`, đợi hai khung hình, thêm `.on`.

6.3. Thanh thời gian

Mọi thanh thời gian dùng một đồng hồ chung. Mẫu từ `vcnvStartTimer()`:

```
var dur = 15;
startObstacleTimer();                                  /* hiện thanh, về 0 */
startClock({
        mode: 'down', duration: dur, label: 'ten_bat_ky',
        onTick: function (remaining) { setObstacleTimer(1 - remaining / dur); },   /* 0 → 1 */
        onDone: function () {
                setObstacleTimer(1);
                window.GameVariables.changed();                /* bắt buộc: màn quản lý cần biết
hết giờ */
        }
});
```



Hình 9. Thanh thời gian: dải đỏ và chấm đỏ (TIMER_DOT = 18 px)

Mỗi lúc chỉ chạy được một đồng hồ; `startClock` mới sẽ huỷ đồng hồ cũ. Thanh mới cần một phần `...Fill` (dải đỏ, đổi `width`) và một `...Dot` (chấm đỏ, đổi `left`); có thể dùng lại class `vcnvTimerFill` / `vcnvTimerDot` như Tăng tốc và Về đích.

6.4. Ví dụ đầy đủ: thêm một đồ hoạ mới có hiệu ứng vào

Giả sử cần một dải chữ "GIẢI LAO" phóng to vào giữa màn hình khi bấm phím G.

Bước 1, `default.html`: thêm phần tử làm con trực tiếp của `.stageContainerDiv` (để F6 tự ẩn nó, xem chương 8):

```
<div class="breakDiv offstage">
        <p class="breakTextP">GIẢI LAO</p>
</div>
```

Bước 2, `Styles/olympia.css`: dùng mẫu A.

```
.breakDiv {
        position: absolute;
        left: 560px; top: 440px; width: 800px; height: 200px;
        transform-origin: 50% 50%;
        transform: scale(1);
        transition: transform 600ms linear;
}
.breakDiv.offstage { transform: scale(0.001); opacity: 0; }
.breakDiv.noTransition { transition: none; }
.breakTextP { font-size: 96px; color: #fff; text-align: center; line-height: 200px; }
```



Hình 10. Vị trí dải "GIẢI LAO" trên sân khấu (minh hoạ, tỉ lệ 1:4)

Bước 3, file mới `Scripts/BreakControl.js`: các hàm hiện / ẩn theo mẫu A và B.

```
function showBreakBanner() {
        var root = document.querySelector('.breakDiv');
        if (root) { root.classList.remove('offstage'); }
        window.GameVariables.IsBreakBannerShowing = true;
}

function hideBreakBanner() {
        var root = document.querySelector('.breakDiv');
        if (!root) { return; }
        root.classList.add('noTransition');
        root.classList.add('offstage');
        void root.offsetWidth;
        root.classList.remove('noTransition');
        window.GameVariables.IsBreakBannerShowing = false;
}

function toggleBreakBanner() {
        if (window.GameVariables.IsBreakBannerShowing) { hideBreakBanner(); } else {
showBreakBanner(); }
}
```

Nạp file trong `default.html`, đặt cạnh các `*Control.js` khác (trước `KeyboardController.js`):

```
<script src="Scripts/BreakControl.js"></script>
```

Bước 4, **`Variables.js`**, khối *Boolean Variables*:

```
gameVariables.IsBreakBannerShowing = false;
```

Bước 5, **`KeyboardController.js`**, trong `handleGlobalKey` (phím dùng ở mọi vòng):

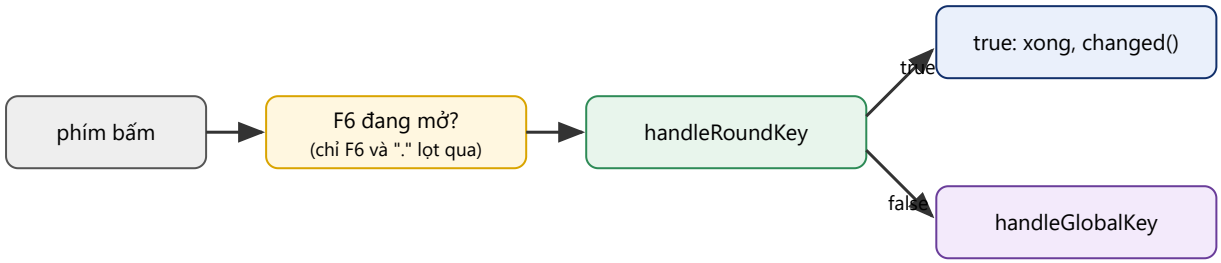
```
if (lower == 'g') { toggleBreakBanner(); return true; }              // key 'g' - "GIẢI LAO"
banner
```

Thêm dòng G vào bảng phím ở đầu file. Nếu dải chữ này phải chặn phím M khi đang hiện, thêm `window.GameVariables.IsBreakBannerShowing ||` vào `areRoundGraphicsOnScreen()` trong `ScoreSummaryControl.js`.

7. Thêm tính năng mới

7.1. Thêm phím mới

Phím dùng ở mọi vòng: thêm vào `handleGlobalKey()`. Phím chỉ dùng trong một vòng: thêm khối vào `handleRoundKey()`, trả về `true` khi vòng đã dùng phím, `false` để phím rơi xuống phím chung (Hình 11).



Hình 11. Đường đi của một phím trong `Keyboard.dispatch`

Nếu phím có tác dụng mặc định của trình duyệt (phím F1-12, dấu cách, mũi tên, Backspace), thêm nó vào `SWALLOWED_KEYS` ở đầu file. Phím giữ lâu bị bỏ qua (`e.repeat`), và phím gõ trong ô nhập liệu của màn quản lý không được tính.

7.2. Thêm nút trên màn quản lý

Có hai trường hợp.

Nút làm đúng việc của một phím có sẵn: chỉ cần HTML trong `control.html`, không viết JavaScript. Thuộc tính `data-cmd` là lệnh phím của vòng (`kdKey`, `vcnvKey`, `ttKey`, `vdKey`), `data-key` là phím:

```
<button type="button" class="bar-btn amber" data-cmd="vdKey" data-key="s"><kbd>S</kbd>Ngôi sao</button>
```

Nút phím chung (không theo vòng) dùng `data-sendkey`, ví dụ `<button class="btn sm" data-sendkey="F6">F6</button>`.

Nút làm việc mới: cần ba chỗ.

1. `Scripts/PanelCommands.js`: đặt tên lệnh và gọi hàm.

   ```
   window.GameVariables.Commands.breakToggle = function () { toggleBreakBanner(); };
   ```

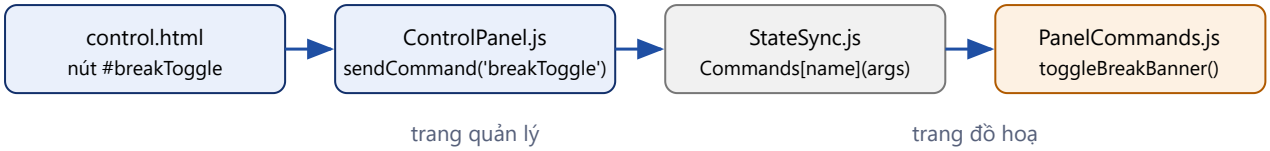
2. `control.html`: thêm nút có `id`.

   ```
   <button type="button" class="btn" id="breakToggle">Giải lao</button>
   ```

3. `Scripts/ControlPanel.js`: gắn sự kiện, đặt cạnh các `addEventListener('click', ...)` khác trong hàm khởi tạo.

   ```
   $('#breakToggle').addEventListener('click', function () { gv.Sync.sendCommand('breakToggle', {}); });
   ```

Lệnh chạy trên `default.html`; màn quản lý chỉ gửi tên lệnh và tham số (đối số thứ hai của `sendCommand`, nhận lại ở `function (args)`).



Hình 12. Đường đi của một lệnh từ nút bấm

7.3. Cho màn quản lý thấy thêm một thông tin

Màn quản lý không đọc thẳng biến trong `Variables.js`; nó đọc bản chụp trạng thái do `PanelStateControl.js` dựng. Thêm một dòng vào đúng vòng trong `buildRoundStateForPanel()`:

```
tt: {
        ...
        showingScores: g.TangTocSequenceCounter == 9,
```

Rồi đọc `tt.showingScores` trong hàm `renderTangToc(tt)` của `ControlPanel.js`. Biến không thuộc vòng nào (như `IsBreakBannerShowing`) thì thêm vào `buildSnapshot()` trong `StateSync.js`, và đọc ở `render(s)` của `ControlPanel.js`.

7.4. Phát âm thanh ở một bước có sẵn

Thêm một dòng gọi hàm âm thanh (bảng ở mục 5.1) vào hàm bước. Âm thanh cần trễ một chút so với hình (như `kd_complete.ogg` phát sau khi đồ hoạ tắt 1 giây):

```
setTimeout(function () { startGeneralSound('ten_file.mp3'); }, 1000);
```

8. Quy tắc để không làm hỏng thứ khác

Mọi file dùng chung một không gian tên. Hàm và biến `var` ở cấp cao nhất của mọi file đều là biến toàn cục. Hai file cùng khai báo `function showBoard()` thì file nạp sau đè file nạp trước, không báo lỗi. Vì vậy:

- Hàm của vòng bắt đầu bằng tên vòng (`vcnv...`, `veDich...`).
- Hàm đồ hoạ nói rõ đồ hoạ nào (`showObstacleBoard`, không phải `showBoard`).
- Hằng số viết hoa kèm tiền tố (`VCNV_TIMER_WIDTH`, không phải `TIMER_WIDTH`).
- Trước khi đặt tên mới, tìm tên đó trong thư mục `Scripts`.

`control.html` cũng nạp `Variables.js`, `StateSync.js` và `KeyboardController.js`, nhưng không nạp các file đồ hoạ và âm thanh. Trong ba file đó, không gọi hàm đồ hoạ ở cấp cao nhất (ngoài hàm), nếu không màn quản lý sẽ báo lỗi ngay khi mở.

Không chạy mã chạm DOM ở cấp cao nhất của file. Khi các file được nạp, trang chưa dựng xong. Việc dựng đồ hoạ làm trong `init()` (`Init.js`).

Chữ đưa lên màn hình dùng `textContent`, không dùng `innerHTML`. Câu hỏi và tên thí sinh đến từ XML và ô nhập liệu; `innerHTML` sẽ biến ký tự `<` trong đề thành thẻ HTML.

Hàm đồ hoạ phải chịu được lúc phần tử chưa có. Theo mẫu có sẵn: `if (!root) { return; }` ở đầu hàm.

Đồ hoạ mới đặt làm con trực tiếp của `.stageContainerDiv`. Khi bảng F6 mở, CSS ẩn mọi con trực tiếp của sân khấu trừ nền và bảng F6, rồi trả lại nguyên trạng khi đóng. Phần tử lồng sâu hơn sẽ không được ẩn.

```
.stageContainerDiv
    ├ .stageBackgroundDiv (luôn hiện)
    ├ .questionBarDiv (bị ẩn khi F6)
    ├ .vcnvDiv, .ttDiv, .vdDiv (bị ẩn khi F6)
    ├ .breakDiv (đồ hoạ mới: đặt ở đây)
    └ .boardScreenDiv (bảng F6)

CSS: .stageContainerDiv.boardMode > div:not(.stageBackgroundDiv):not(.boardScreenDiv) { ẩn }
```

Hình 13. Chỉ con trực tiếp của sân khấu mới được F6 tự ẩn

Mọi thay đổi trạng thái xảy ra ngoài phím bấm phải gọi `window.GameVariables.changed()` để màn quản lý cập nhật. Sau mỗi phím và mỗi lệnh từ màn quản lý thì việc này tự động; bạn chỉ cần tự gọi trong `setTimeout`, `onDone` của đồng hồ, hoặc sự kiện `ended` của âm thanh / video.

Đổi số bước của vòng thì tìm hết chỗ dùng số đó (mục 5.4, bước 4).

9. Kiểm tra sau khi sửa (không bắt buộc)

1. Kiểm tra cú pháp, nếu máy có Node.js (mở terminal ở thư mục dự án):

   ```
   node --check Scripts/KeyboardController.js
   ```

   Không in gì là không có lỗi cú pháp. Muốn kiểm tất cả: `for f in Scripts/*.js Scripts/Rounds/*.js; do node --check "$f"; done` (Git Bash).
2. Mở `default.html` và `control.html`, bấm F12 trên cả hai, xem tab Console có dòng đỏ nào không. Dòng đỏ ghi tên file và số dòng lỗi.
3. Chạy lại phần liên quan trong `CHECKLIST.md` (danh sách bước kiểm tra từng vòng), bằng cả phím lẫn nút trên màn quản lý.
4. Khi bấm một phím mà không có gì xảy ra, gõ vào Console tên bộ đếm của vòng (ví dụ `window.GameVariables.TangTocSequenceCounter`) rồi so với nhánh trong `KeyboardController.js` để biết phím đó có được xử lý ở bước này không.

10. Lỗi thường gặp

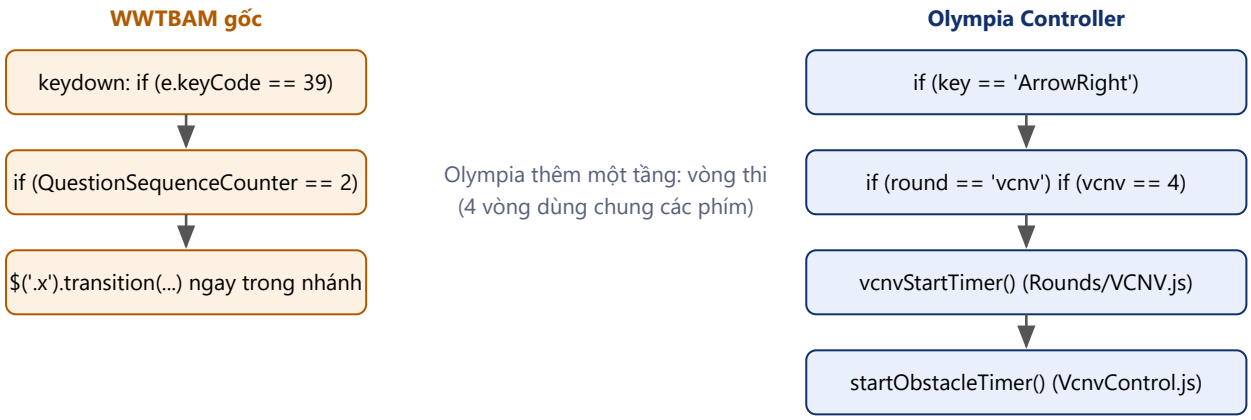
Triệu chứng	Nguyên nhân hay gặp
Console báo `xxx is not defined`	Gọi hàm chưa có, gõ sai tên, hoặc quên thêm thẻ `<script>` của file mới vào `default.html`
Bấm phím không có gì xảy ra	Bộ đếm không ở bước mà nhánh chờ; bảng F6 đang mở (khoá mọi phím trừ F6 và dấu chấm); con trỏ đang ở trong ô nhập liệu
Một chức năng cũ bỗng chạy sai sau khi thêm hàm mới	Trùng tên hàm toàn cục (chương 8)
Đồ hoạ hiện nhưng không có hiệu ứng	Thêm `.on` và `.in` cùng lúc (mẫu C) hoặc thiếu `transition` trong CSS
Đồ hoạ ẩn đi vẫn có hiệu ứng	Thiếu dòng `void root.offsetWidth;` (mẫu B)
Màn quản lý không cập nhật	Thay đổi xảy ra trong `setTimeout` / `onDone` mà không gọi `window.GameVariables.changed()`; hoặc thông tin mới chưa được thêm vào `PanelStateControl.js`
Âm thanh không phát	Tên file sai (phân biệt hoa thường trên một số máy chủ); trình duyệt chặn âm thanh khi chưa bấm phím nào trên trang `default.html`; file chưa có trong `Sounds/`
Âm thanh phát hai lần	Đang mở hai tab `default.html`
Sửa xong mà trang vẫn chạy như cũ	Trình duyệt dùng bản cũ: tải lại bằng Ctrl+F5

11. Đối chiếu với WWTBAM HTML5 Controller gốc

Thư mục `WWTBAM Original Project` chứa bộ điều khiển "Ai là triệu phú" mà Olympia Controller lấy làm khuôn. Chương này giúp người đã quen bộ đó tìm đúng chỗ tương ứng. Số dòng đếm bằng `wc -l` trên hai thư mục `Scripts` tại thời điểm viết tài liệu.

WWTBAM gốc	Olympia Controller	Ghi chú
`Variables.js` (196 dòng)	`Variables.js` (394)	Cùng khuôn một IIFE, chia khối theo kiểu biến. Olympia có thêm bảng bộ đếm và các hàm dùng chung ở cuối file.
`KeyboardController.js` (1326), 25 nhánh `e.keyCode ==`	`KeyboardController.js` (303)	Cùng ý: một chỗ cho mọi phím. Gốc so `keyCode` và viết thẳng lệnh jQuery trong nhánh; Olympia so `e.key`, nhánh chỉ gọi hàm bước nằm trong `Rounds/*.js`.
`QuestionSequenceCounter`, `MoneyTreeSequenceCounter` …	`KhoiDongSequenceCounter`, `VcnvSequenceCounter`, `TangTocSequenceCounter`, `VeDichSequenceCounter`	Một bộ đếm cho mỗi vòng.
`AudioControl.js` (198)	`AudioControl.js` (180)	Giữ tên hàm của gốc: `startGeneralSound`, `startLongPassiveSound` … Olympia thêm nhạc nền có đoạn lặp.
`TimerControl.js` (933)	`TimerControl.js` (60)	Olympia chỉ có một đồng hồ chung cho mọi thanh thời gian.
`Init.js` (285)	`Init.js` (183)	Cùng cảnh báo "DO NOT TAMPER". Gốc đọc XML bằng `$.ajax` đồng bộ; Olympia dùng `fetch` bất đồng bộ và đọc năm file XML.
`GeneralControl.js` (89)	`GeneralControl.js` (61)	Gốc: logo, âm thanh theo cấp câu hỏi. Olympia: chuyển vòng.
`QuestionAndAnswerControl.js` (308)	`QuestionControl.js`, `TangTocControl.js`, `VeDichControl.js`, `VcnvControl.js`	Mỗi vòng Olympia có kiểu thanh câu hỏi riêng.
`MoneyTreeControl.js` (357)	`ScoreboardControl.js`, `ScoreBoardScreenControl.js`	Thang tiền thưởng được thay bằng thanh điểm 4 thí sinh và bảng F6.
`WinningsControl.js` (139)	`ScoreSummaryControl.js`	Hiện số tiền thắng được thay bằng tổng kết điểm (M).
`ContestantsControl.js` (22)	`getContestantsForGame()` trong `Init.js`	Gốc có 1 người chơi, Olympia có 4.
`FastestFingerControl.js` (144)	`VcnvTreeControl.js`	Cây đáp án Olympia gần với màn "nhanh tay" của gốc: nhiều người trả lời, xếp theo thời gian.
`LifelineControl.js` (911)	Ngôi sao hy vọng trong `VeDichControl.js` / `Rounds/VeDich.js`	Olympia chỉ có một "quyền trợ giúp".

`CameraControl.js`, `jquery-2.1.1.min.js`, `jquery.3d.js`, `accounting.js`	không có	Olympia không dùng jQuery; thao tác DOM bằng `document.querySelector` và `classList`.
không có	`StateSync.js`, `ControlPanel.js`, `PanelStateControl.js`, `PanelCommands.js`, `control.html`	Màn quản lý thứ hai là phần riêng của Olympia; gốc chỉ điều khiển bằng bàn phím trên chính trang đồ hoạ.



Hình 14. Cùng một mẫu "phím → bộ đếm → lệnh" ở hai dự án



Hình 15. Thanh câu hỏi của bộ gốc (`WWTBAM Original Project/Images/Large_Strap_Unedited.png`), tương ứng thanh câu hỏi Khởi động ở Hình 8